

DBMS -PAYMENT MANAGEMENT
PROJECT TITLE-SHOPWITHNYKAA
DBMS - WEEK 5

1. Database Design
Payment Table

Attribute	Constraint
Payment_ID	Primary Key
Order_ID	Foreign Key
Payment_Mode	NOT NULL
Payment_Date	NOT NULL
Payment_Amount	NOT NULL
Payment_Status	NOT NULL

2. Table Relationship

- **Orders.Order_ID → Primary Key**
- **Payment.Payment_ID → Primary Key**
- **Payment.Order_ID → Foreign Key referencing Orders.Order_ID**
- **Relationship: One-to-Many (1:M) — one order can have payment records.**

3. Create Payment Table

```
SQL> CREATE TABLE Payment (  
2     Payment_ID NUMBER PRIMARY KEY,  
3     Order_ID NUMBER,  
4     Payment_Mode VARCHAR2(20) NOT NULL,  
5     Payment_Date DATE NOT NULL,  
6     Payment_Amount NUMBER(10,2) NOT NULL,  
7     Payment_Status VARCHAR2(20) NOT NULL,  
8     CONSTRAINT fk_payment_order  
9         FOREIGN KEY (Order_ID)  
10        REFERENCES Orders(Order_ID)  
11 );
```

Table created.

4. Insert 5 Payment Records

```
SQL> INSERT INTO Payment VALUES
  2 (701, 1, 'UPI', DATE '2026-09-02', 1600.00, 'Successful');

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2 (702, 2, 'Credit Card', DATE '2026-09-03', 2300.00, 'Successful');

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2 (703, 3, 'Debit Card', DATE '2026-09-05', 1750.00, 'Failed');

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2 (704, 4, 'UPI', DATE '2026-09-08', 3200.00, 'Successful');

1 row created.

SQL>
SQL> INSERT INTO Payment VALUES
  2 (705, 5, 'Cash on Delivery', DATE '2026-09-10', 950.00, 'Pending');

1 row created.
```

5. Manage payment transactions

5.1 Display successful payments

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Successful';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
701	1	UPI	02-SEP-26	1600	Successful
702	2	Credit Card	03-SEP-26	2300	Successful
704	4	UPI	08-SEP-26	3200	Successful

5.2 Display failed payments

```
SQL> SELECT * FROM Payment WHERE Payment_Status = 'Failed';
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT	PAYMENT_STATUS
703	3	Debit Card	05-SEP-26	1750	Failed

6. Modify payment status

```
SQL> UPDATE Payment
  2 SET Payment_Status = 'Successful'
  3 WHERE Payment_ID = 703;

1 row updated.

SQL>
SQL> SELECT * FROM Payment WHERE Payment_ID = 703;
```

PAYMENT_ID	ORDER_ID	PAYMENT_MODE	PAYMENT_D	PAYMENT_AMOUNT
703	3	Debit Card	05-SEP-26	1750
Successful				

7. Analyze payment method

```
SQL> SELECT
  2 Payment_Mode,
  3 COUNT(*) AS Payment_Count,
  4 SUM(Payment_Amount) AS Total_Amount
  5 FROM Payment
  6 WHERE Payment_Status = 'Successful'
  7 GROUP BY Payment_Mode
  8 ORDER BY Payment_Mode;
```

PAYMENT_MODE	PAYMENT_COUNT	TOTAL_AMOUNT
Credit Card	1	2300
Debit Card	1	1750
UPI	2	4800

8. Payment transaction method

```
SQL> SELECT
  2 o.Order_ID,
  3 o.Customer_ID,
  4 o.Order_Date,
  5 p.Payment_ID,
  6 p.Payment_Mode,
  7 p.Payment_Date,
  8 p.Payment_Amount,
  9 p.Payment_Status
 10 FROM Orders o
 11 JOIN Payment p
 12 ON o.Order_ID = p.Order_ID
 13 ORDER BY o.Order_ID;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	PAYMENT_ID	PAYMENT_MODE	PAYMENT_D
1	101	02-SEP-26	701	UPI	02-SEP-26
2	102	03-SEP-26	702	Credit Card	03-SEP-26
3	103	05-SEP-26	703	Debit Card	05-SEP-26
4	104	08-SEP-26	704	UPI	08-SEP-26
5	105	10-SEP-26	705	Cash on Delivery	10-SEP-26

ORDER_ID	CUSTOMER_ID	ORDER_DAT	PAYMENT_ID	PAYMENT_MODE	PAYMENT_D
1	101	02-SEP-26	701	UPI	02-SEP-26
2	102	03-SEP-26	702	Credit Card	03-SEP-26
3	103	05-SEP-26	703	Debit Card	05-SEP-26
4	104	08-SEP-26	704	UPI	08-SEP-26
5	105	10-SEP-26	705	Cash on Delivery	10-SEP-26